

Peripherals (Easy Explanation)

Peripherals are devices that are connected to the **CPU** but are **not part of the CPU** itself.

Examples of peripherals:

- Hard disk
- Mouse
- Keyboard
- Printer
- CD/DVD drive

Important points for exam:

- Peripherals are **slow** compared to the CPU.
- Because they are slow, they can **delay the CPU**.
- The CPU often has to **wait** for them.

Example (easy to remember):

When you play a game and move to the **next level**, the game data is:

- **Read from the CD** (slow device)
- **Written into memory**
During this time, the CPU must wait.

Interface (Easy Explanation)

What is an Interface?

An **interface** is a **bridge** between the **CPU** and **peripherals**.

It allows the CPU and peripherals to **communicate properly**, even though they are very different.

Interface = Hardware + Software

Hardware part

- The physical connection
- Example: **USB port**, cable socket

Software part

- A **driver** program
- Example: **Printer driver** installed from a CD or website

📌 **Exam line to remember:** An interface is the combination of hardware and software that connects peripherals to the CPU.

Why Interfaces Are Needed

Peripherals are all different. They may:

- Work at **different speeds**
- Use **different data transfer rates**
- Use **different codes and control signals**
- Send data in **serial** or **parallel** form
- Use **higher voltage** than the CPU
- Always work **slower than the CPU**

So, the interface makes everything **compatible**.

Main Functions of an Interface

You should remember these points:

1. **Buffering**
2. **Serial ↔ Parallel conversion**
3. **Analogue ↔ Digital conversion**
4. **Voltage conversion**
5. **Protocol conversion**
6. **Handling status signals**

Buffering

What is Buffering?

- A **buffer** is a small area of **RAM** inside the interface.
- It **temporarily stores data** while data moves between CPU and peripheral.

Why buffering is needed?

- CPU is **fast**
- Peripherals are **slow**
- The buffer stores data so the CPU does **not have to wait**

Simple example:

Think of a **water tank**:

- Slow tap fills the tank
- Fast motor uses water from the tank
- 👉 The tank is like the **buffer**

📌 **Exam line:** Buffering compensates for speed differences between CPU and peripherals.

Serial and Parallel Data Transmission

Data transmission: It means **sending data from one device to another**.

Serial Data Transmission

- Data is sent **one bit at a time**
- Bits are sent **in sequence**

Features:

- **Slow**
- **Good for long distances**
- Uses **fewer wires**

Example: USB, mouse, keyboard

Parallel Data Transmission

- **Multiple bits** sent at the same time
- Often **16 or 32 bits together**

Features:

- **Very fast**
- Only good for **short distances**
- Uses **many wires**

Example: Internal CPU buses

Why Conversion Is Needed

- The CPU works with **parallel data**
- Many peripherals send **serial data**
- The interface:
 1. **Buffers** serial data
 2. **Converts serial** → **parallel**
 3. Sends it to the CPU

Exam line: Serial data from peripherals is converted into parallel data before being sent to the processor.

Interfaces: Analogue and Digital Data

Analogue Signals

Many real-world electrical signals are **analogue**. An analogue signal **changes smoothly** between two values and it is **continuous**, not fixed. **Important point:** Peripherals (like microphone, sensors) often send **analogue signals**. If you could see it, it would look like a **wave** going up and down.

Digital Signals

Computers can **only understand digital signals**. Digital signals have **only two values**, **ON = 1** and **OFF = 0**.

Example of a digital signal: 0 1 1 0 0 1 1

📌 **Exam line:** Digital signals consist of a series of 1s and 0s.

ADC and DAC

Why ADC and DAC are needed

The CPU works with **digital data**, while many peripherals work with **analogue data**, so conversion is required.

DAC – Digital to Analogue Converter

- Converts **digital signals** → **analogue signals**

- Used when the **computer sends data to a peripheral**

Example:When a computer sends sound to speakers, digital sound is converted into analogue sound.

✂ Exam line: A DAC converts digital signals from the computer into analogue signals for peripherals.

ADC – Analogue to Digital Converter

- Converts **analogue signals** → **digital signals**
- Used when a **peripheral sends data to the computer**

Example:When a microphone sends sound to a computer, analogue sound is converted into digital data.

✂ Exam line: An ADC converts analogue signals into digital signals that the CPU can process.

Voltage Conversion

Peripherals use **different voltage levels**, while the CPU and motherboard use **lower voltage**. Direct connection can cause **errors or damage**.

Role of interface

- Adjusts voltage levels
- Makes signals safe and compatible

✂ Exam line: Voltage conversion ensures that peripherals and CPU can communicate safely.

Protocol Conversion

A **protocol** is a set of **rules** that controls how data is **sent**, **received**, and **understood**.

Protocol Conversion

Different devices use different protocols. The interface converts protocols so the **peripheral understands the computer** and the **computer understands the peripheral**.

Exam line:Protocol conversion allows different devices to communicate correctly.

Handling of Status Signals

Status signals tell the system whether a device is **ready**, **busy**, or has an **error**.

Example: When a printer shows “Out of paper”, the computer knows the printer is not ready.

Exam line: Status signals inform the user and CPU about the condition of a peripheral.

Buffers and Spoolers

Buffer

A buffer is a **small area of memory (RAM)** used to **temporarily store data**. It helps data transfer between the CPU and peripheral.

Why buffers are used

The CPU is **very fast** and peripherals are **slow**, so the buffer balances the speed difference.

Examples: Printer buffer, Keyboard buffer (stores keystrokes)

 **Exam line:** A buffer provides temporary storage and compensates for speed differences.

Advantages of Buffers

- CPU can send data **quickly** to buffer
- Peripheral reads data **slowly** from buffer
- CPU is **not interrupted frequently**

Spoolers

In spooling, data is first stored on **secondary storage (hard disk)** and later sent to the peripheral when the CPU is free. It is also called **background processing**.

In **printer spooling**, multiple print jobs are stored on disk and printed one by one.

Exam line: Spooling stores data on disk and sends it to slow peripherals in the background.

Buffer vs Spooler

Buffer

- Uses **RAM**
- Limited size
- Faster access

Spooler

- Uses **hard disk**

- Very large storage
- Can be slow if CPU is very busy

Important exam point: Using both buffer and spooler gives the best system performance.

Current Interface Trends

New interfaces are **always being developed**. Technology is improving to make computers **faster and easier to use**.

Main goals of new interfaces

- **Increase speed**
So data moves faster between CPU and peripherals
- **Support wireless communication**
So devices can connect **without cables**

Exam line: Modern interface development focuses on higher speed and wireless communication.

Increasing Interface Speeds

How interface speed is measured

Speed is measured in **Megabits per second (Mbps)**.

Important exam warning:

Megabits (Mb) $\neq$ Megabytes (MB).

8 bits = 1 byte.

Why companies use Mbps: Mbps gives bigger numbers. Looks better in advertisements .

Exam line: Interface speeds are measured in Mbps, not MBps.

Examples of Faster Interfaces

USB Interfaces

- **USB 1** → Maximum speed **12 Mbps**
- **USB 2** → Speed increased to **480 Mbps**
- That is **40 times faster** than USB 1

FireWire Interfaces

- **FireWire 400** → 400 Mbps
- **FireWire 800** → 800 Mbps (double the speed)

Exam line: Newer interfaces provide much higher data transfer rates than older versions.

Interface Standards

A **standard** is an agreed method of connection. It ensures devices work together correctly.

Common Interface Standards

Interface	Full Form / Use
RS232	Serial communication standard
SCSI	Parallel interface for devices
IDE	Integrated Drive Electronics
SATA	Serial ATA (up to 1.5 Gbps)
IEEE	Standards body (e.g. FireWire)
MIDI	Musical instrument interface
PCI	Peripheral Component Interconnect
PCMCIA	Laptop expansion cards
USB 1 & 2	Universal Serial Bus

Why standards are important

Computer manufacturers follow standards. Peripheral manufacturers follow standards. Devices can work together easily.

Interface standards make devices **compatible**.

Latest Interfaces

USB 1

Used for external devices. Speed: **12 Mbps**.

USB 2

Improved version of USB. Speed: **up to 480 Mbps**.

FireWire (IEEE 1394)

FireWire is a high-speed serial interface. It is used for digital cameras and audio/video equipment. FireWire 1394b has a speed of **800 Mbps**, and future versions can reach **up to 3.2 Gbps**.

Wireless Communication

Data is sent **without cables**. It uses **radio waves**.

Advantages

- No need for line-of-sight
- Signals pass through walls
- Devices don't need to face each other

Exam comparison: Wireless radio waves are better than infrared because they can pass through walls.

Bluetooth

Bluetooth is used for short-range communication and connects personal devices.

Examples

- Mobile phone ↔ headset
- Laptop ↔ mouse
- Laptop ↔ keyboard

Features

- Short distance
- Low power
- Speed expected up to **2 Mbps**

 **Exam line:** Bluetooth is used for short-range wireless communication between personal devices.

WiFi

WiFi stands for **Wireless Fidelity**. It is used for **wireless networking**.

Wireless Local Area Network (**WLAN**).

Features

- Range: **15–50 meters**
- Speed: **5–20 Mbps**
- Used in homes, schools, and offices

Exam line: WiFi allows devices to connect to a network wirelessly over short distances.

Bluetooth vs WiFi

Bluetooth	WiFi
Short range	Longer range
Low power	Higher power
Personal devices	Network access
Keyboard, mouse	Internet access

Solid-State Storage Devices

What is a Solid-State Storage Device?

A solid-state storage device has **no moving parts**. Data is stored **electronically**, not mechanically, which makes these devices faster and more reliable.

Examples

- Flash cards
- USB flash memory (pen drive)

 **Exam line:** Solid-state storage devices contain no moving parts.

Flash ROM (Backing Storage Medium)

Both flash cards and USB flash memory use **flash ROM** as the storage medium. The **main difference** between them is that they use **different interfaces** to connect to computers or peripherals.

Flash Cards

Uses

Flash cards are mainly used in **digital cameras**. They can store photos, videos, programs, and other data files.

Important points

There are **many different standards** of flash cards. A digital camera usually supports **only one type** of flash card. **Card readers** are used to read different types of flash cards on a computer.

All-in-One Devices

Some printers and scanners have **built-in card slots**. This allows **direct printing** from a flash card without using a computer.

 **Exam line:** Flash cards are commonly used for data storage in digital cameras.

USB Flash Memory

Types

USB flash memory devices are based on their interface type:

- USB 1
- USB 2

Important point

Most USB flash drives today support **USB 2**, which is **much faster** than USB 1.

📌 **Exam line:** USB flash memory is portable and uses the USB interface.

Features & Advantages of Solid-State Storage Devices

1. Small Size

Flash cards are small enough to fit **inside digital cameras**. USB flash drives are also very small and can fit on **key rings** or even **watches**.

2. Strong and Robust

Because solid-state devices have **no moving parts**, they are **less likely to be damaged**. They are not affected by shaking or movement.

Example:

iPods use solid-state storage.

📌 **Exam line:** Solid-state devices are robust because they have no moving parts.

3. Low Power Consumption

Solid-state storage devices use **less power** than hard disks. This allows batteries to last **longer**.

Example:

Music players can play for longer periods using solid-state storage.

4. Different Storage Capacities

Solid-state storage devices are available in different sizes, ranging from **256 MB to 8 GB** (at the time).

5. Replacement of Floppy Disks

USB flash memory has **replaced floppy disks** because it is portable, has a higher storage capacity, and works with most computers.

6. Security Uses

Some USB flash drives include **fingerprint recognition** or special security software. In some systems, the computer will **not start** unless the USB device is present.

 **Exam line:** USB flash memory is also used for security applications.

Developments in Backing Storage Devices

1. Increased Capacity – DVD

A **single-layer DVD-R** can store **4.7 GB** of data, while a **dual-layer DVD-R** can store **8.4 GB**. Most modern computers support **dual-layer DVDs**.

2. Blu-ray Disc

Blu-ray is a **next-generation optical disc** designed for **high-definition (HD) video** and large data storage.

 **Exam line:** Blu-ray discs support high-definition video and large storage capacity.

3. Increased Read & Write Speeds

Newer and faster interfaces have been developed, such as:

- USB 2 → **480 Mbps**
- FireWire 800 → **800 Mbps**

4. Reduced Physical Size

Backing storage devices are becoming **smaller** over time. For example, Toshiba introduced a **0.85-inch hard disk**, which was the smallest in 2004.

5. Lower Cost per Unit of Storage

The cost of storage has **decreased significantly** over time.

Example (for exam):

- 1995 → 100 MB hard disk = £300
- 2005 → 100 MB ZIP = £5
- 512 MB USB flash = £25

 **Exam line:** The cost per unit of storage has decreased over time.

Implications of Storage Developments

Compatibility Issues

New storage devices may **not be compatible** with older computers. For example, older PCs may not have USB ports, USB 1 is slower than USB 2, and older DVD drives may not support newer formats.

Archived Data Problem

Old storage formats can become **unreadable** over time. To prevent data loss, information must be **copied regularly** to newer storage formats.

Real-Life Example

VHS video recorders were replaced by **DVD players**, and modern digital video cameras now record directly to **DVDs**. Exam line: Technological development may cause compatibility problems with older hardware.

Types of Computers (Easy Explanation)

There are **four main types of computers**, classified according to their **size, speed, and processing power**:

1. Mainframe Computer
2. Super Computer
3. Mini Computer
4. Micro Computer

1. Mainframe Computer

A mainframe computer is a **very large and very powerful** computer. It works at **very high speed** and can handle **huge amounts of data**.

Main Features

- Large data word size, usually **64-bit or more**
- **Very large memory capacity**
- Can support **many users simultaneously**

Uses

- Military defense systems
- Large business data processing
- Computer graphics display systems

Example

- **IBM 4381**

 **Exam line:**

A mainframe computer is a large, powerful computer used for large-scale data processing.

2. Super Computer

A super computer is the **fastest and most powerful** type of computer. It is even more powerful than a mainframe computer.

Uses

- Scientific research
- Weather forecasting
- Government organizations
- Universities

Example

- **Cray Y-MP / 832**

 **Exam line:** Supercomputers are used for complex calculations requiring extremely high speed.

3. Mini Computer

A mini computer is a **smaller version of a mainframe computer**. It is **less powerful**, has **slower speed**, and uses **less memory** than a mainframe.

Uses

- Scientific research
- Industrial control systems

 **Exam line:** A mini computer is a scaled-down version of a mainframe computer.

4. Micro Computer

A micro computer is the **smallest type of computer**. Its CPU is usually a **single microprocessor**.

Examples

- Desktop computer
- Laptop computer
- Palmtop computer

 **Exam line:**Microcomputers are small computers that use a single microprocessor as CPU.

Microprocessor Data Types (Easy Explanation)

A microprocessor can work with **different types of data**.

Common Data Types

- Bit
- Byte
- Word
- Unsigned binary integer
- Signed binary integer
- BCD numbers
- ASCII
- Floating-point numbers

1. Bit

A bit is the **smallest unit of data**. It can have only one of two values:

- 0
- 1

 **Exam line:**A bit is the smallest unit of information.

2. Byte

A byte is formed by combining **8 bits**.

 **Exam line:**A byte consists of 8 bits.

3. Word

A word is the **amount of data a CPU can process at one time**. The word size depends on the type of processor.

Examples

- 8-bit word
- 16-bit word
- 32-bit word

 **Exam line:** A word is the basic data size processed by a microprocessor.

Unsigned and Signed Binary Integers

Unsigned Binary Integer

An unsigned binary integer has **no sign**. It stores **only positive values**, and the leading bit is part of the value.

Example

- Memory addresses

 **Range formula:** Unsigned range = 0 to $(2^n - 1)$

Examples

- 5-bit unsigned → 0 to 31
- 8-bit unsigned → 0 to 255

Signed Binary Integer

A signed binary integer can store **both positive and negative numbers**. The **most significant bit (MSB)** shows the sign:

- 0 → positive
- 1 → negative

Positive numbers are stored in **true binary**, while negative numbers are stored using **two's complement**.

Forms of Signed Binary Numbers

(a) Sign-Magnitude Form

- MSB indicates the sign
- Has **two zeros**: +0 and -0

📌 **Range:** $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$

(b) 1's Complement Form

- Negative number is formed by **inverting all bits**
- Has **two zeros**

📌 **Range:** $-(2^{n-1} - 1)$ to $+(2^{n-1} - 1)$

(c) 2's Complement Form (Most Important)

- Negative number = **1's complement + 1**
- Has **only one zero**
- Most widely used representation

📌 **Range:** $-(2^{n-1})$ to $+(2^{n-1} - 1)$

📌 **Exam line:** Two's complement representation is widely used because it has only one zero.

8-bit Number System (Very Important)

Unsigned: Range: 0 to 255

Signed

- Positive range: **0 to +127**
- Negative range: **-1 to -128**

BCD (Binary Coded Decimal) Numbers

What is BCD?

In BCD, each **decimal digit** is represented by a **4-bit binary number**.

Forms of BCD

1. **Unpacked BCD**
 - One digit stored in one byte
2. **Packed BCD**
 - Two digits stored in one byte

Exam line: In BCD, each decimal digit is represented by a 4-bit binary number.

ASCII (Easy Explanation)

ASCII stands for **American Standard Code for Information Interchange**. It is a **character coding system** used by computers.

How ASCII works

Each character is represented by an **integer value**. The computer stores this integer value in its memory.

What ASCII represents

- **Alphabets** such as A–Z and a–z
- **Digits** from 0 to 9
- **Special symbols** like @, #, \$, %, etc.

Example

- Character '**A**' has ASCII value **65**
- Character '**a**' has ASCII value **97**
- Character '**0**' has ASCII value **48**

✂ **Exam line:** ASCII represents alphanumeric characters and special symbols using integer codes in memory.

Floating-Point Numbers (Easy Explanation)

Floating-point numbers are used to represent **very large numbers**, **very small numbers**, and **fractional values**.

Components of a Floating-Point Number

A floating-point number consists of **three main parts**:

1. Sign
2. Exponent
3. Mantissa (fraction)

✂ **Exam line:** A floating-point number consists of sign, exponent, and mantissa.

Example (Decimal Floating-Point)

Consider the number -2.5×10^{-2} .

- **Sign** → Negative
- **Exponent** → -2
- **Mantissa** → 2.5

Binary Floating-Point Number

A binary floating-point number is written as a **normalized binary fraction multiplied by a power of 2**.

Normalized form: In normalized form, there is only one bit (1) before the binary point.

Example- 1.0101×2^3

 **Exam line:** A binary floating-point number is stored as a normalized binary fraction raised to a power of 2.

Floating-Point Conversion Procedure

Steps to convert a decimal number to binary floating-point

1. Convert the **absolute value** of the number into binary
 - Convert integer and fractional parts separately
2. Multiply the result by 2^0
3. **Normalize** the binary number
4. Store the bits after the leading 1 as the **mantissa**
5. Add **bias** to the exponent and store it
6. Set the **sign bit**
 - 0 for positive
 - 1 for negative

Bias Formula (Important)

Bias is calculated using the formula:

$$\text{Bias} = 2^{(k-1)} - 1$$

where **k** is the number of exponent bits.

Examples

- **8-bit floating-point** $\rightarrow$ Bias = **3**
- **IEEE 32-bit** $\rightarrow$ Bias = **127**
- **IEEE 64-bit** $\rightarrow$ Bias = **1023**

Worked Example (Easy Explanation)

Convert 2.625 to 8-bit floating-point

1. Convert to binary
 - $2 = 10_2$
 - $0.625 = 0.101_2$
 - Combined result $\rightarrow 10.101_2$
2. Normalize
 - 1.0101×2^1
3. Mantissa $\rightarrow 0101$
4. Exponent
 - $1 + 3 \text{ (bias)} = 4 = 100_2$
5. Sign bit $\rightarrow 0$

Final Answer **0 100 0101**

Hexadecimal representation: **45₁₆**

IEEE 32-bit (Single Precision)

- Sign $\rightarrow$ **1 bit**
- Exponent $\rightarrow$ **8 bits**
- Mantissa $\rightarrow$ **23 bits**
- Bias $\rightarrow$ **127**

IEEE 64-bit (Double Precision)

- Sign $\rightarrow$ **1 bit**
- Exponent $\rightarrow$ **11 bits**

- Mantissa → **52 bits**
- Bias → **1023**

📌 **Exam line: IEEE 754 standard defines the format for floating-point representation.**

1. Major Parts of a Computer System

A **microprocessor-based system** consists of four main parts that work together to perform all computer operations.

1) CPU (Central Processing Unit)

- The CPU is the **brain of the computer system**.
- It performs all **thinking, calculations, and control operations**.

2) Memory

- Memory stores **programs (instructions)** and **data** that are needed by the CPU.

3) Input / Output (I/O) Circuitry

- I/O circuitry allows the computer to **receive input** from external devices and **send output** to external devices.

4) Buses

Buses are **groups of wires** that connect different parts of the computer system and allow communication between them.

There are **three types of buses**:

- Address Bus
- Data Bus
- Control Bus

2. Memory (Easy Explanation)

Memory is used to store information required for processing.

Functions of memory

- Stores **binary codes of instructions (programs)**
- Stores **binary data**

Types of memory

- **ROM (Read Only Memory)** – stores permanent data
- **RAM (Random Access Memory)** – stores temporary data
- **Magnetic / Optical disks** – such as hard disk, CD, and DVD

✂ **Exam line:** Memory stores instructions and data in binary form.

3. Input / Output (I/O)

Input and output devices allow communication between the computer and the external world.

What is I/O?

- Input devices **send data into** the computer
- Output devices **send data out** of the computer

Connection of I/O devices

- I/O devices are connected to the microprocessor through **I/O ports**

Examples

- Input devices: Keyboard, mouse
- Output devices: Monitor, printer
- Input/Output device: Modem

✂ **Exam line:** I/O devices allow communication between the computer and the outside world.

4. Central Processing Unit (CPU)

The CPU controls and executes all operations in the computer system.

Functions of CPU

- Controls **all operations of the computer**
- Fetches instructions from memory
- Decodes instructions
- Executes instructions step-by-step

Main steps performed by CPU

1. **Fetch** the instruction from memory

2. **Decode** the instruction
3. **Execute** the instruction

Important parts inside CPU

- General-purpose registers
- Special-purpose registers
- Control signal generating circuits

 **Exam line:** CPU controls, processes, and executes instructions.

5. Registers (Very Important for Exam)

Registers are **small, high-speed storage locations inside the CPU** used to store data temporarily.

(a) General Purpose Registers (8085)

- The 8085 microprocessor has **six 8-bit general-purpose registers**
- The registers are named: **B, C, D, E, H, L**
- These registers store **temporary data**

Register pairs (16-bit)

- BC
- DE
- HL

These register pairs are used when **16-bit operations** are required.

(b) Special Purpose Registers

1) Accumulator (A)

- It is an **8-bit register**
- It is part of the **Arithmetic Logic Unit (ALU)**
- It stores the **result of arithmetic and logic operations**

 **Exam line:** Accumulator stores the final result of ALU operations.

2) Flag Register

The flag register stores information about the **status of the result** produced by the ALU.

Main flags

- **Sign Flag (S)** – indicates positive or negative result
- **Zero Flag (Z)** – indicates result is zero
- **Parity Flag (P)** – indicates even or odd number of 1s
- **Carry Flag (CY)** – indicates carry or borrow
- **Overflow Flag (OF)** – indicates signed arithmetic error

(c) Memory Registers

- **Program Counter (PC)** – stores the address of the next instruction
- **Stack Pointer (SP)** – points to the top of the stack

6. ALU, Signed and Unsigned Numbers

The Arithmetic Logic Unit performs all arithmetic and logic operations.

Important points

- ALU performs **binary arithmetic only**
- ALU does not know whether a number is signed or unsigned
- ALU only sets the **status flags**

Signed Numbers

- The **most significant bit (MSB)** is used as the **sign bit**
- 0 indicates a positive number
- 1 indicates a negative number

Unsigned Numbers

- All bits represent the value
- There is **no sign bit**

7. Carry Flag (CY)

The carry flag is mainly used in **unsigned arithmetic**.

Addition

- Carry flag is set when a carry is generated from the MSB

Example:

$$1111 + 0001 = 0000$$

Carry = 1

Subtraction

- Carry flag is set when a borrow is required

Example:

$$0000 - 0001 = 1111$$

Carry = 1

8. Overflow Flag (OF)

Overflow flag is used in **signed arithmetic**.

Conditions for overflow

Case 1: Positive + Positive = Negative

$$0100 + 0100 = 1000$$

Overflow = 1

Case 2: Negative + Negative = Positive

$$1000 + 1000 = 0000$$

Overflow = 1

 **Important point:** Mixed sign addition never causes overflow.

Example Explanation

Example 1

$$0111 + 0010 = 1001$$

- Unsigned interpretation:
 - $7 + 2 = 9$ ✓
 - Carry flag is considered
- Signed interpretation:
 - $7 + 2 = -7$ ✗
 - Overflow flag indicates error

Example 2

$$1111 + 0001 = 0000$$

- Unsigned interpretation:
 - $15 + 1 = 0$ ❌
 - Carry flag indicates error
- Signed interpretation:
 - $-1 + 1 = 0$ ✓
 - Overflow flag is not set

9. Address Bus

- Address bus is **unidirectional**
- Direction: CPU → Memory / I/O
- Size may be 16, 20, 24, 32, or 36 lines
- It carries the **address** of memory or I/O location

📌 **Exam line:** More address lines allow access to more memory.

10. Data Bus

- Data bus is **bidirectional**
- Size may be 8, 16, or 32 lines
- Transfers **data** between CPU, memory, and I/O devices
- Only **one device** can use the data bus at a time

11. Control Bus

- Control bus has **4 to 10 control lines**
- It carries **control signals**. Examples-
 - Memory Read
 - Memory Write

📌 **Exam line:** The control bus tells devices what operation to perform.

12. Memory System

Every microprocessor-based system must include memory.

Main types of memory

- **ROM** – permanent storage
- **RAM** – temporary storage

13. Memory Pin Connections

All memory chips have four types of pins.

Types of pins

- Address pins
- Data pins
- Selection pins
- Control pins

Address Pins

- Labeled as A0 to An
- A0 is the least significant address bit
- A **1K memory** requires **10 address pins (A0–A9)**

Data Pins

- Used for reading and writing data

Examples

- **16K × 1** → 16K memory locations, 1 bit each
- **64K × 4** → total storage of 256K bits

Selection Pins

- Known as: CS (Chip Select), CE (Chip Enable). Used to enable or disable the memory chip

Control Pins

ROM

- Uses **OE (Output Enable)**
- Data appears at output only when:
 - CS is active

- OE is active

RAM

- Uses one control pin:
 - R/W
- Or two control pins:
 - WE (Write Enable)
 - OE (Output Enable)

⚠ Important rule: WE and OE must **never be active at the same time**.

ADDRESS DECODING

Address decoding means selecting the correct memory chip using the address sent by the microprocessor. When the microprocessor places an address on the address bus, many memory chips may be connected, but only one memory chip must respond to that address. Address decoding makes sure that the correct chip is selected and all other chips remain inactive.

Why is address decoding needed?

Address decoding is needed so that each memory chip works in a unique part of the memory map. If address decoding is not used, then only one memory device can be connected to the microprocessor, and the system becomes almost useless because multiple memory chips cannot be distinguished.

Exam line: Address decoding ensures that only one memory device responds to a given address.

MAIN MEMORY ARRAY DESIGN

Main Memory Array Design means connecting many small memory chips together to form a bigger memory system. Since a single memory chip usually cannot provide the required memory size, several chips are combined in a systematic way to meet the total memory requirement of the system.

Addressing capacity

A 16-bit microprocessor can generate 16 address lines. This means it can address ($2^{16} = 65,536$) memory locations, which is equal to 64K memory locations.

M/IO signal

The M/IO signal tells whether the microprocessor is accessing memory or an I/O device. When M/IO is HIGH, the operation is a memory operation. When M/IO is LOW, the operation is an I/O operation. This signal helps in enabling the correct device during an operation. This signal tells whether the CPU is accessing memory or I/O.

MEMORY DECODING TECHNIQUES

To connect ROM or RAM with a microprocessor, the following memory decoding techniques are used:

- Linear decoding
- Full decoding
- Partial decoding
- Memory decoding using PALs / PLDs

1. Linear Decoding

In linear decoding, memory chips are selected directly using address lines without using any extra decoding hardware. The address lines are connected in a simple way to enable memory chips.

Example

A 4K SRAM can be made using four 1K SRAM chips. This memory arrangement can be expanded to 6K by using higher address lines such as A14 and A15.

Advantage

Linear decoding is very simple to design and does not require any decoder IC.

Disadvantages

In linear decoding, bus conflict can occur. If two address lines go LOW at the same time, more than one memory chip may get selected, causing data collision on the bus. Memory foldback is another problem where the same memory appears at multiple addresses, which wastes address space.

✂ **Solution:** Use full address decoding.

2. Full Address Decoding

Meaning

In full address decoding, all address lines are used to decode memory. This ensures that every memory location has one unique address and only one memory chip responds to a particular address.

Advantages

Full decoding avoids bus conflict and memory foldback. It uses the address space efficiently and is the safest and most reliable decoding technique.

Exam line: In full decoding, each memory location is selected by a unique address.

3. Partial Address Decoding

Meaning

In partial address decoding, only some of the address lines are used for decoding. As a result, the complete address space is not fully decoded.

Result

Due to partial decoding, one physical memory location may correspond to multiple addresses. This causes address repetition in the memory map.

Used when

Partial decoding is used when only a part of the total memory space is required and full decoding is not necessary.

Exam line: Partial decoding uses fewer address lines and causes address repetition.

DECODER (Important)

A decoder is a combinational logic circuit that has n input lines and (2^n) output lines. When the decoder is enabled, only one output line becomes HIGH for a specific input combination. Decoders are used to detect a specific binary code. Decoder detects a specific binary code.

2-to-4 Decoder

Inputs

- A1
- A0
- Enable (E)

Outputs

- Y0
- Y1
- Y2
- Y3

Working

When the enable input E is set to 1, only one output becomes HIGH depending on the values of A1 and A0. This property is used to select one memory chip at a time. Used to select memory chips.

MEMORY DESIGN USING DECODERS

Full Decoding using Decoder

In full decoding, a decoder such as a 3×8 decoder is used. Address lines like A12, A11, and A10 are connected to the decoder inputs. The decoder is enabled only when the correct control signals are active, ensuring that only one SRAM chip is selected for a given address. This ensures that only one chip is selected.

PROGRAMMABLE LOGIC DEVICES (PLDs)

What are PLDs?

Programmable Logic Devices are ICs that contain AND gates and OR gates. These devices can be programmed electrically and are commonly used for address decoding and logic design. This programming is hardware programming, not software programming.

Types of PLDs

- PROM (Programmable ROM)
- PAL (Programmable Array Logic)
- PLA (Programmable Logic Array)

1. PROM (Programmable ROM)

PROM has a fixed AND array and a programmable OR array. It can be programmed only once. In PROM, a decoder generates all possible minterms, and the OR gates select the required minterms to produce the output. Output is the sum of minterms.

2. PAL (Programmable Array Logic)

PAL has a programmable AND array and a fixed OR array. It generates only the required product terms, making it faster and simpler than PROM. Output is the sum of products.

3. PLA (Programmable Logic Array)

PLA has both programmable AND and programmable OR arrays. It is the most flexible PLD and allows sharing of common product terms. PLA is used for complex logic designs. Output is the sum of products.